Exercise 1

First Code and Binary Representation

Released: 21 October 2026 | Due: 28 October 2026, 23:59

Learning objectives

In this exercise, you will construct seven-segment digit patterns and use an 8×8 LED matrix to display and transform binary representations.

General instructions

The teaching assistant responsible for this exercise is Itay Shedlezki.

The exercise has two parts. In Part 1, you will construct the binary encodings of decimal digits and display them in sequence. In Part 2, you will read values, examine their binary representations, manipulate individual bits, convert between signed representations, and implement shift and rotation animations. You must complete and submit both `ex1-digits.c` and `ex1-matrix.c`.

Clone the public course repository at github.com/intro2cs-biu/ex1 and work on your local copy:

```
# Clone the repository
git clone https://github.com/intro2cs-biu/ex1.git

# Enter the exercise directory
cd ex1
```

You cannot push changes to the course repository. If you copy the repository to your own GitHub account, that copy must be private. Making solutions publicly accessible is strictly prohibited because other students may copy them, which can lead to an academic-integrity review for everyone involved.

Please post questions about the exercise **publicly** in the course forum.

Read the Coding Style file on the course site and work accordingly. Code that does not follow it may receive a point reduction. Compile with `-Wall -Wextra -Werror` and resolve every warning and error. Testing on the departmental server is highly recommended. An exercise containing warnings or errors may receive a penalty of up to 100 points.

Submit by **28 October 2026 at 23:59** to **both** the course submission system (one ZIP containing the two C files) and the Google Form (the two C files uploaded directly, not inside a ZIP). See the Submission section for full instructions.

Late submissions are penalized as follows:

- One day late: 10 points. The maximum grade is 90.
- Two days late: 20 points.
- Three days late: 30 points.
- **You cannot submit this or any other exercise more than three days late.**

Requests for an extension, for justified reasons only, must be sent to intro2cs.biu@gmail.com and include:

- full name,
- submission-system username, and
- ID number

Important

The purpose of this introductory course is not only to learn C syntax. More importantly, it is to develop **logical thinking, algorithmic thinking, problem-solving skills, and the ability to debug your own programs**. Getting stuck, reading an error, reconsidering an algorithm, asking a question, and trying again are essential parts of that learning process.
AI code generation is especially tempting when you are stuck, but allowing it to finish a function, generate an algorithm, or fix a bug removes precisely the work through which these skills develop. Therefore, do not use generative-AI systems to produce, complete, rewrite, or debug your exercise, even if you intend to read or modify the generated result afterward.
Your submitted work must be written by you. Do not share code with other students. You are reminded that thinking together is a blessing. Working together is cheating. An exercise identified as copied will receive **0** for everyone involved, and additional action may be taken.

Required language and operations

Implement this exercise only with material covered in class: variables, assignment, the course input/output operations, and bitwise operations. Do not use loops, `if`, `switch`, conditional operators, arithmetic operators, or other language and library features not used in class.

Specifically, throughout this assignment, use only bitwise operators for computations. Arithmetic operators such as `+`, `-`, `*`, `/`, and `%` are forbidden, including increment (`++`), decrement (`--`), and arithmetic compound assignments (such as `+=` and `-=`).

The Intro2CS library and its documentation describe many capabilities. For this assignment, only the operations presented in class and required below are permitted, regardless of what else the library provides.

Work with the Arduino board

The physical board is the primary environment for this exercise. Build each circuit exactly as shown, compile the corresponding source file, send it to the Arduino, and check the connected display. Disconnect USB power before changing any wire.

Keep the exercise and library folders next to each other:

```
intro2cs/
|-- ex1/
|   |-- ex1-digits.c
|   `-- ex1-matrix.c
`-- intro2cs-lib/
    |-- headers/
    |-- intro2cs-arduino-lib.a
    `-- intro2cs-arduino-simulator-lib.a
```

For either source file, replace `PROGRAM` below with `ex1-digits` or `ex1-matrix`:

```
# Compile the program
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os \
  -std=c99 -Wall -Wextra -Werror \
```

```
  -I../intro2cs-lib/headers PROGRAM.c \
  ../intro2cs-lib/intro2cs-arduino-lib.a -o PROGRAM.elf

# Create the Arduino HEX file
avr-objcopy -O ihex -R .eeprom PROGRAM.elf PROGRAM.hex

# Send the HEX file to the Arduino
avrdude -p atmega328p -c arduino -P PORT -b 115200 -D \
  -U flash:w:PROGRAM.hex:i
```

Replace `PORT` with the port reported by `arduino-cli board list`, for example `COM3` on Windows, `/dev/ttyACM0` on Linux, or `/dev/cu.usbmodem...` on macOS. Close any serial monitor before sending a new program. Reopen it at 9600 baud when the program uses serial input or output.

Exact serial formatting

Read all input and output instructions carefully and follow the attached examples exactly. The automatic grading system compares your output with the expected output character by character. Your output must therefore be **exactly the same** as the examples, including capitalization, punctuation, spaces, tabs, and newlines.

A score of 100 from the automatic submission system is **not the final grade**. The automatic check is used only to verify the general input and output structure. The actual evaluation uses substantially more extensive tests, including additional inputs and edge cases, and also checks compliance with the exercise requirements.

Part 1 – digits display: `ex1-digits.c`

First disconnect the board and build the circuit exactly as shown in Figure 1.

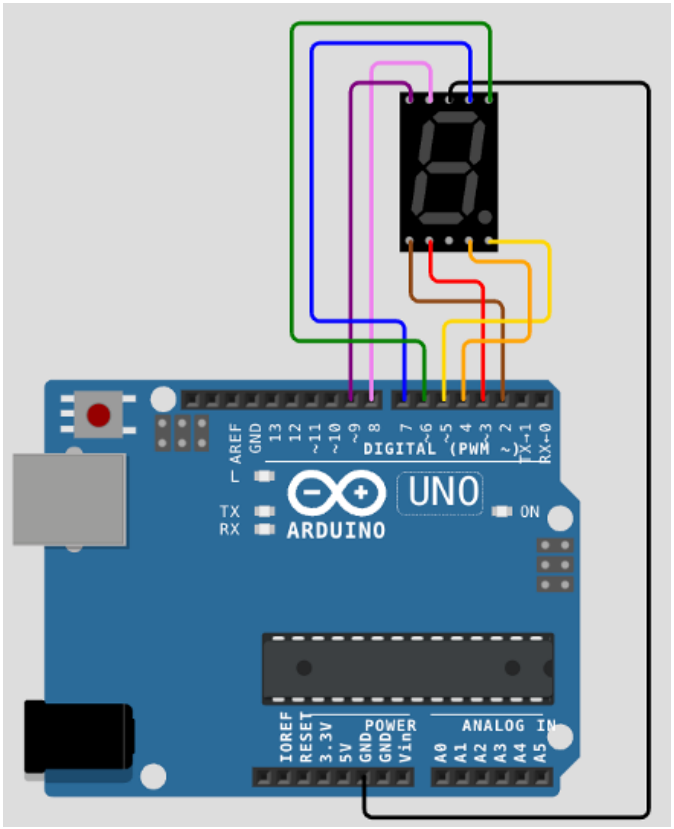


Figure 1: Required wiring for the seven-segment display.

A digit is formed from seven bars named A through G and an optional decimal point named DP, as shown in Figure 2.

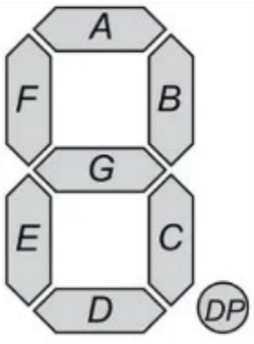


Figure 2: Names of the seven segments and decimal point.

Their states are stored in an `unsigned char` using this order:

Bit	7	6	5	4	3	2	1	0
Segment	DP	G	F	E	D	C	B	A

Complete `ex1-digits.c` so that the display shows the decimal digits 0 through 9 in increasing order. Each digit must remain visible for exactly 1000 milliseconds before the next digit appears. Determine the ten binary segment patterns yourself using the diagram and bit order above.

Compile, send, and run `ex1-digits` on the Arduino. Confirm that every digit is recognizable and appears in the required order before continuing.

Example

Input: none.
Output: `0` → `1` → `2` → `3` → `4` → `5` → `6` → `7` → `8` → `9`, with each digit displayed for one second.

Part 2 – Binary representation on the LED matrix: `ex1-matrix.c`

Disconnect the board and build the exact circuit shown in Figure 3 before writing or testing the matrix program.

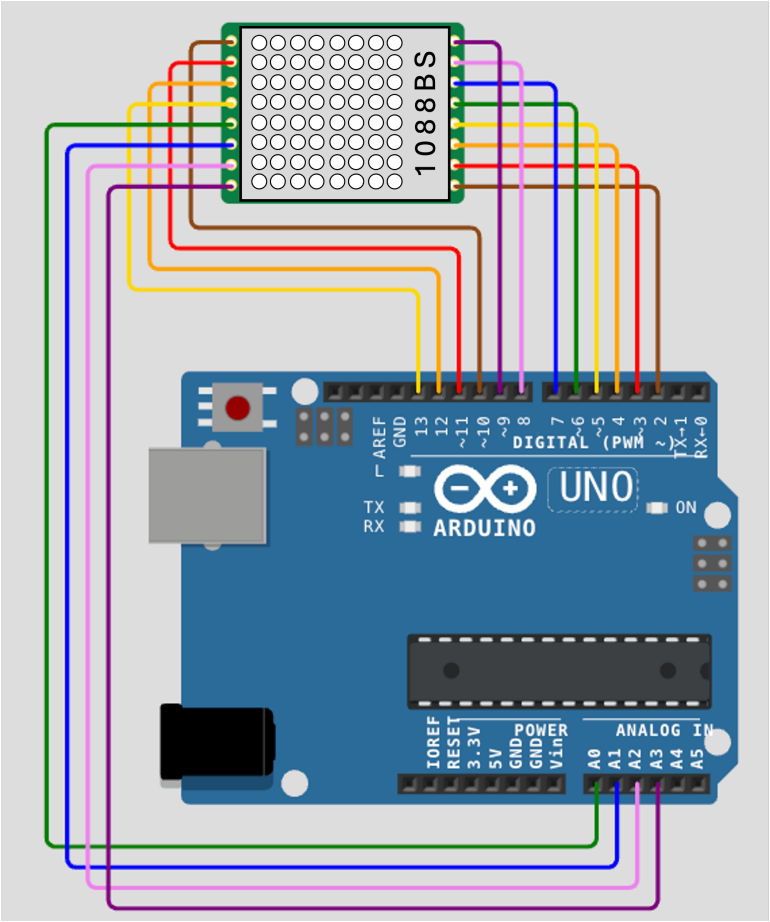


Figure 3: Required wiring for the 8×8 LED matrix.

The matrix has eight rows of eight pixels. Rows and columns are numbered from 0 to 7. Row 0 is the top row and row 7 is the bottom row. Column 0 is the leftmost column and column 7 is the rightmost column. Therefore, pixel `(0, 0)` is at the top-left corner and pixel `(7, 7)` is at the bottom-right corner.

The course library lets a program control the display at three different levels, using a value whose type matches the amount of information being changed:

- **One pixel:** `matrixSetPixel(row, col, value)` receives a position and a binary value: `0` turns the pixel off and `1` turns it on.
- **One row:** `matrixSetRowFromByte(row, value)` receives a row number and an `unsigned char`. Its eight bits control the row's eight pixels. Bit 7 is the leftmost pixel and bit 0 is the rightmost pixel.
- **The complete matrix:** `matrixSetFrom64Bits(frame)` receives a full 8×8 frame as an `unsigned long long`. Its 64 bits control all eight rows, with the most significant byte at the top and the least significant byte at the bottom.

Before controlling the display, call `matrixBegin(NULL, NULL)` once. Use `matrixClear()` to turn off all 64 pixels. In this exercise, the only permitted matrix functions are these two operations and the three pixel, row, and complete-frame functions described above.

The library also provides functions that control the display using arrays. We will learn that interface later in the course, when arrays are introduced. Do not use it in this exercise.

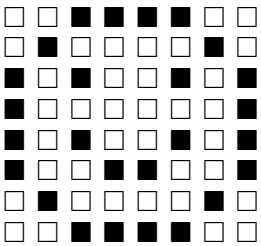
Recall that on the Arduino Uno, an `int` is represented using two bytes. To represent a complete eight-byte frame, use an `unsigned long long`.

Read the matrix-library documentation for full details about these functions and how to use them.

Initialize the matrix and serial communication at the beginning of the program. Preserve the task headings in the starter file and produce all messages specified in this document exactly. Capitalization, spaces, punctuation, and newlines are part of the required output.

Task 0 – first frame

Use an `unsigned long long` to store the following face. Clear the matrix and then display the frame. Black squares are lit pixels and white squares are unlit pixels.



The same face must appear on the physical matrix. If it does not, compare every connection with Figure 3 and correct the wiring before continuing.

Example

Input: none.
Exact serial-monitor output:

```
------ Task 0: Smile Face ------
```

The face shown above appears on the matrix.

Task 1 – one value in three bases

Clear the matrix and prompt the user to enter a character: `Please enter a character:\n`. Read one character and print that character followed by its decimal, lowercase hexadecimal, and octal character-code representations using the exact headings and text specified above.

Example

Input: `A`
Exact serial-monitor interaction:

```
------ Task 1: Different Representations ------
    Please enter a character:
    A
    You entered the character: A
    In decimal: 65
    In hexadecimal: 41
    In octal: 101
```

Task 2 – inspect and change one bit

Clear the matrix and prompt the user to enter a signed one-byte value: `Please enter a number between -128 and 127:\n`, and then a bit position: `Please enter a position between 0 and`

`7:\n`. Position 0 is the least-significant bit. Display these four rows. Set row 0 pixel by pixel and each of rows 1–3 with one library call.

1. the original value in two's-complement representation
2. the value with the selected bit flipped ($0 \to 1, 1 \to 0$)
3. the value with the selected bit cleared (set to 0)
4. the value with the selected bit set (set to 1)

All transformations must be expressed with bitwise operations.

Example

Input: value `-42` (`11010110`), position `1`
Exact serial-monitor interaction:

```
------ Task 2: Bit Manipulation ------
    Please enter a number between -128 and 127:
    -42
    Please enter a position between 0 and 7:
    1
```

Matrix rows 0–3:

```
11010110
11010100
11010100
11010110
```

Task 3 – complements and alternating masks

Clear the matrix and prompt the user to enter a signed one-byte value: `Please enter a number between -127 and 127:\n`. Task 3 accepts values from -127 through 127. In an eight-bit one's-complement representation, one bit stores the sign, so -128 cannot be represented. Sign-magnitude representation has the same limitation. Display the following four rows, using a single library call for each row:

1. the original value in two's-complement representation
2. the one's-complement representation of the same signed value
3. the sign-magnitude representation of the same signed value
4. if the original value is odd, a filtered row that clears every even-numbered bit position, otherwise a filtered row that clears every odd-numbered bit position

Determine and apply the required masks using bitwise operations only. Do not use arithmetic or conditional statements.

For the input `0`, all four rows must contain `00000000`. Treat zero as a nonnegative, even value: its two's-complement, one's-complement, and sign-magnitude representations are all positive zero, and filtering it leaves the row unchanged.

Hint. Left-shifting signed values in C may result in undefined behavior, so you may want to store the value in an unsigned variable before shifting it.

Example

Input: `-37`, whose two's-complement byte is `11011011`
Exact serial-monitor interaction:

```
------ Task 3: Complements Representation ------
    Please enter a number between -127 and 127:
    -37
```

Matrix rows 0–3:

```
11011011
11011010
10100101
10001010
```

Task 4 – shift animation

Clear the matrix and prompt the user to enter an 8×8 frame using a 64-bit hexadecimal representation: `Please enter frame in hexadecimal:\n`. Read the frame using `serialReadUnsignedLongLong` from `serial.h` with base `16`, and store it in an `unsigned long long`. Display the frame, then shift every row one pixel to the left once per second. Pixels leaving the left edge disappear and must not enter a neighboring row. Display each of the eight shifted frames, including the final empty frame.

Example

Input: `0103070F1F3F7FFF`, which forms a pyramid from the top row to the bottom row.
Exact serial-monitor interaction:

```
------ Task 4: Left Shift Animation ------
    Please enter frame in hexadecimal:
    0103070F1F3F7FFF
```

First two and last two displayed frames:

```
Frame 1          Frame 2                  Frame 8          Frame 9
00000001         00000010                 10000000         00000000
00000011         00000110                 10000000         00000000
00000111         00001110                 10000000         00000000
00001111         00011110        ...      10000000         00000000
00011111         00111110                 10000000         00000000
00111111         01111110                 10000000         00000000
01111111         11111110                 10000000         00000000
11111111         11111110                 10000000         00000000
```

Task 5 – rotate animation

Clear the matrix and prompt the user to enter another 8×8 frame using a 64-bit hexadecimal representation: `Please enter frame in hexadecimal:\n`. Read the frame in the same way as in Task 4. Once per second, rotate every row one pixel to the left. A pixel leaving the left edge must reappear at the right edge of the same row. Display the rotations until the frame returns to its original state, then display the final "Done" message exactly as written in the starter file.

Example

Input: `0103070F1F3F7FFF`, which forms a pyramid from the top row to the bottom row.
Exact serial-monitor interaction:

```
------ Task 5: Circular Left Shift Animation ------
    Please enter frame in hexadecimal:
    0103070F1F3F7FFF


------Done! ------
```

First two and last two displayed frames:

```
Frame 1          Frame 2                Frame 8          Frame 9
00000001         00000010               10000000         00000001
00000011         00000110               10000001         00000011
00000111         00001110               10000011         00000111
00001111         00011110       ...     10000111         00001111
00011111         00111110               10001111         00011111
00111111         01111110               10011111         00111111
01111111         11111110               10111111         01111111
11111111         11111111               11111111         11111111
```

Complete input and output example

Appendix A presents one complete interaction. Each input value appears in blue directly after the prompt where it is entered.

Compile, send, and run `ex1-matrix` on the Arduino. Check the displayed pixels after every task and verify the serial interaction at 9600 baud.

Verify output with the simulator

After both programs work on the physical board, use the simulator when you need to compare their textual output precisely. From the `ex1` folder on Linux, compile a file as follows, replacing `PROGRAM` with `ex1-digits` or `ex1-matrix`:

```
# Compile the simulator program
gcc -std=c99 -Wall -Wextra -Werror -no-pie \
  -I../intro2cs-lib/headers PROGRAM.c \
  ../intro2cs-lib/intro2cs-arduino-simulator-lib.a \
  -o PROGRAM

# Run the simulator program
./PROGRAM
```

On macOS, use the installed GNU GCC command, such as `gcc-16`, and omit `-no-pie`. On Windows, use the Windows GCC command, backslashes in paths, omit `-no-pie`, and give the executable an `.exe` suffix.

Compare the required text character by character. Do not add debugging output to the submitted files.

Reference programs for both parts are included in the repository. Choose the `executables/macos`, `executables/linux`, or `executables/windows` folder. Each contains simulator executables for both parts together with the corresponding Arduino `.hex` files. Run the simulator program for your

operating system to understand the required behavior and output. These programs are references only. Do not include them in the submission ZIP.

Automatic comparison with the supplied test files

The repository's `tests` folder contains input and expected-output files. Do not edit them. From the exercise directory on macOS, Linux, or the departmental server, test the matrix program with:

```
# Run the matrix program with the supplied input
./ex1-matrix < tests/ex1-matrix-input.txt > actual-output.txt

# Compare the actual and expected output
diff -u tests/ex1-matrix-expected-output.txt actual-output.txt
```

The `<` operator gives the program input from the supplied file. The `>` operator saves everything printed by the program and simulator in `actual-output.txt`. `diff` prints nothing when the files are identical. Otherwise, it shows the differing lines.

The digits program needs no input:

```
# Run the digits program
./ex1-digits > actual-digits-output.txt

# Compare the actual and expected output
diff -u tests/ex1-digits-expected-output.txt actual-digits-output.txt
```

On Windows, run the equivalent commands in Command Prompt and use `fc` instead of `diff` to compare the files.

Submission

You may develop your code in any environment. You must know how to take the C source files from that environment, copy them to the departmental server, compile them there, and submit them to both systems listed below. Testing both files on the server is highly recommended to ensure there are no warnings or errors.

Submit only the files containing your code, not files created by an editor, IDE, compiler, operating system, or simulator. **DO NOT** use Hebrew characters in file or folder names. A file whose path contains Hebrew characters cannot be copied reliably to the university servers.

Your submission must contain these two source files, with their original names:

- `ex1-digits.c`
- `ex1-matrix.c`

You must submit your solution to both systems:

1. Course submission system: upload one ZIP archive whose top level contains exactly the two C files above.
2. Google Form: upload the two C files directly, **not inside a ZIP archive**.

Submit the same versions of both C files to both systems. You may submit multiple times; only your last submission in each system will be considered. If you update your solution, resubmit the updated files to both systems so that your final submissions match.

Do not include the library directory, simulator archive, executables, object files, ELF files, HEX files, editor settings, or enclosing folders. Open the ZIP after creating it and confirm its contents. Both source files must compile without warnings on the departmental Linux server.

- ☐ Both required C files are present and keep their original names.
- ☐ Both programs compile with `-Wall -Wextra -Werror`.
- ☐ Both programs were sent to the Arduino and tested with the required physical displays.
- ☐ Any personal GitHub repository containing my work is private.
- ☐ The submission ZIP contains only the two required source files.
- ☐ The same final C files were submitted to both systems: inside a ZIP in the course submission system and directly in the Google Form.

Complete input and output interaction

This appendix shows one complete simulator interaction. Text in blue is entered by the user and is not program output. Each input is shown immediately after the prompt that requests it. Every `[matrix]` block contains all eight matrix rows. `#` is a lit pixel and a space is an unlit pixel.

```
------ Task 0: Smile Face ------
[matrix]
|        |
|        |
|        |
|        |
|        |
|        |
|        |
|        |
[matrix]
|  ####  |
| #    # |
|# #  # #|
|#      #|
|# #  # #|
|#  ##  #|
| #    # |
|  ####  |


------ Task 1: Different Representations ------
[matrix]
|        |
|        |
|        |
|        |
|        |
|        |
|        |
|        |
    Please enter a character:
A
    You entered the character: A
    In decimal: 65
    In hexadecimal: 41
    In octal: 101

------ Task 2: Bit Manipulation ------
[matrix]
|        |
|        |
|        |
|        |
|        |
```

```
|        |
|        |
|        |
    Please enter a number between -128 and 127:
-42
    Please enter a position between 0 and 7:
1
[matrix]
|        |
|        |
|        |
|        |
|        |
|        |
|        |
|        |
[matrix]
|      # |
|        |
|        |
|        |
|        |
|        |
|        |
|        |
[matrix]
|     ## |
|        |
|        |
|        |
|        |
|        |
|        |
|        |
[matrix]
|     ## |
|        |
|        |
|        |
|        |
|        |
|        |
|        |
[matrix]
|   # ## |
|        |
|        |
|        |
|        |
|        |
|        |
|        |
[matrix]
|   # ## |
|        |
|        |
|        |
|        |
|        |
|        |
|        |
[matrix]
| # # ## |
|        |
|        |
|        |
|        |
|        |
|        |
|        |
[matrix]
|## # ## |
|        |
|        |
|        |
```

```
|        |
|        |
|        |
|        |
[matrix]
|## # ## |
|## # #  |
|        |
|        |
|        |
|        |
|        |
|        |
[matrix]
|## # ## |
|## # #  |
|## # #  |
|        |
|        |
|        |
|        |
|        |
[matrix]
|## # ## |
|## # #  |
|## # #  |
|## # ## |
|        |
|        |
|        |
|        |


------ Task 3: Complements Representation ------
[matrix]
|        |
|        |
|        |
|        |
|        |
|        |
|        |
|        |
    Please enter a number between -127 and 127:
-37
[matrix]
|## ## ##|
|        |
|        |
|        |
|        |
|        |
|        |
|        |
[matrix]
|## ## ##|
|## ## # |
|        |
|        |
|        |
|        |
|        |
|        |
[matrix]
|## ## ##|
|## ## # |
|# #  # #|
|        |
|        |
|        |
|        |
|        |
[matrix]
|## ## ##|
|## ## # |
```

```
|# #  # #|
|#   # # |
|        |
|        |
|        |
|        |


------ Task 4: Left Shift Animation ------
[matrix]
|        |
|        |
|        |
|        |
|        |
|        |
|        |
|        |
    Please enter frame in hexadecimal:
0103070F1F3F7FFF
[matrix]
|       #|
|      ##|
|     ###|
|    ####|
|   #####|
|  ######|
| #######|
|########|
[delay] 1000 ms
[matrix]
|      # |
|     ## |
|    ### |
|   #### |
|  ##### |
| ###### |
|####### |
|####### |
[delay] 1000 ms
[matrix]
|     #  |
|    ##  |
|   ###  |
|  ####  |
| #####  |
|######  |
|######  |
|######  |
[delay] 1000 ms
[matrix]
|    #   |
|   ##   |
|  ###   |
| ####   |
|#####   |
|#####   |
|#####   |
|#####   |
[delay] 1000 ms
[matrix]
|   #    |
|  ##    |
| ###    |
|####    |
|####    |
|####    |
|####    |
|####    |
[delay] 1000 ms
[matrix]
|  #     |
| ##     |
|###     |
|###     |
```

```
|###     |
|###     |
|###     |
|###     |
[delay] 1000 ms
[matrix]
| #      |
|##      |
|##      |
|##      |
|##      |
|##      |
|##      |
|##      |
[delay] 1000 ms
[matrix]
|#       |
|#       |
|#       |
|#       |
|#       |
|#       |
|#       |
|#       |
[delay] 1000 ms
[matrix]
|        |
|        |
|        |
|        |
|        |
|        |
|        |
|        |


------ Task 5: Circular Left Shift Animation ------
[matrix]
|        |
|        |
|        |
|        |
|        |
|        |
|        |
|        |
    Please enter frame in hexadecimal:
0103070F1F3F7FFF
[matrix]
|       #|
|      ##|
|     ###|
|    ####|
|   #####|
|  ######|
| #######|
|########|
[delay] 1000 ms
[matrix]
|      # |
|     ## |
|    ### |
|   #### |
|  ##### |
| ###### |
|####### |
|########|
[delay] 1000 ms
[matrix]
|     #  |
|    ##  |
|   ###  |
|  ####  |
| #####  |
|######  |
```

```
|###### #|
|########|
[delay] 1000 ms
[matrix]
|    #   |
|   ##   |
|  ###   |
| ####   |
|#####   |
|#####  #|
|##### ##|
|########|
[delay] 1000 ms
[matrix]
|   #    |
|  ##    |
| ###    |
|####    |
|####   #|
|####  ##|
|#### ###|
|########|
[delay] 1000 ms
[matrix]
|  #     |
| ##     |
|###     |
|###    #|
|###   ##|
|###  ###|
|### ####|
|########|
[delay] 1000 ms
[matrix]
| #      |
|##      |
|##     #|
|##    ##|
|##   ###|
|##  ####|
|## #####|
|########|
[delay] 1000 ms
[matrix]
|#       |
|#      #|
|#     ##|
|#    ###|
|#   ####|
|#  #####|
|# ######|
|########|
[delay] 1000 ms
[matrix]
|       #|
|      ##|
|     ###|
|    ####|
|   #####|
|  ######|
| #######|
|########|


------Done! ------
```